
Distributed Systems

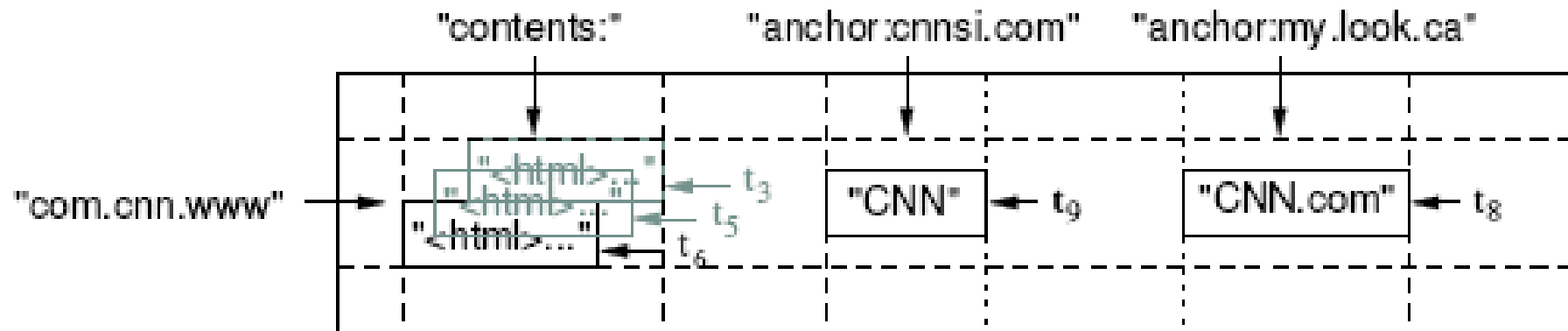
Monsoon 2025

Lecture 24a

International Institute of Information Technology

Hyderabad, India

Google BigTable – Data Model



- A Bigtable is a sparse, distributed, persistent multidimensional sorted map.
- <Row, Column, Timestamp> triple for key - lookup, insert, and delete API
- Arbitrary “columns” on a row-by-row basis
 - Column family:qualifier. Fewer families, many qualifiers is the recommended form.
 - Column-oriented physical store- rows are sparse!
- Does not support a relational model
 - No table-wide integrity constraints
 - No multirow transactions

BigTable Data Model

Column Family 1:
Demographics

Column Family 2:
Identity

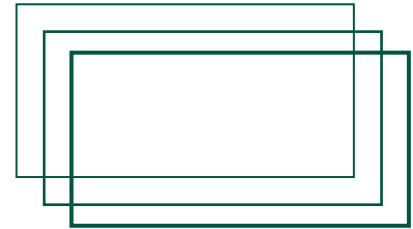
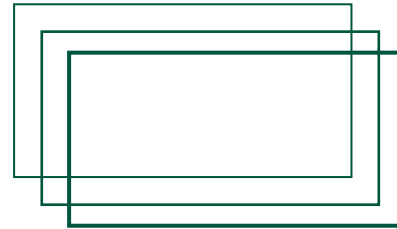
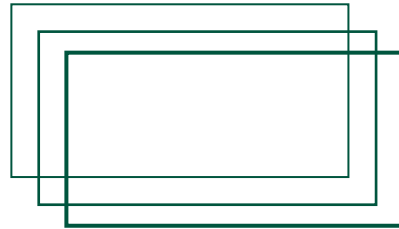
Column Family 3:
Education

c1	c2	c3	c4
----	----	----	----

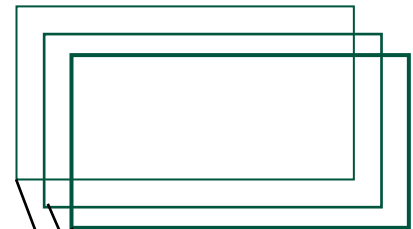
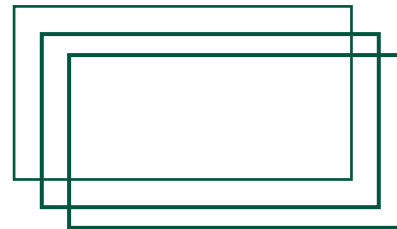
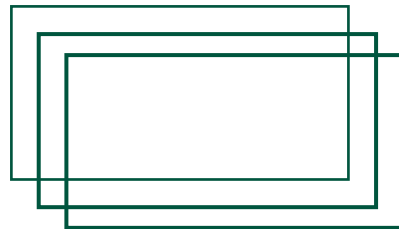
c1	c2	c3	c4
----	----	----	----

c1	c2	c3	c4
----	----	----	----

Row Key 1:
Akash

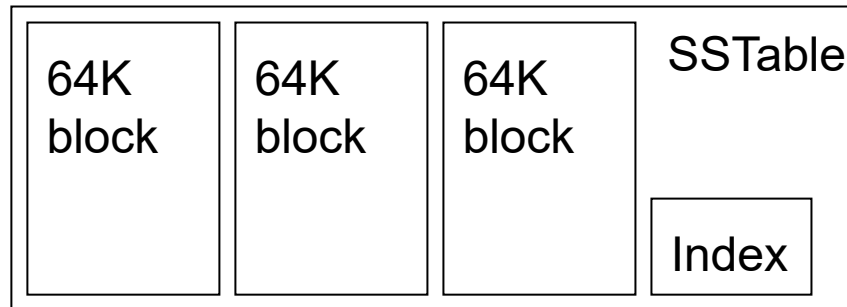


Row Key 2:
Aruna



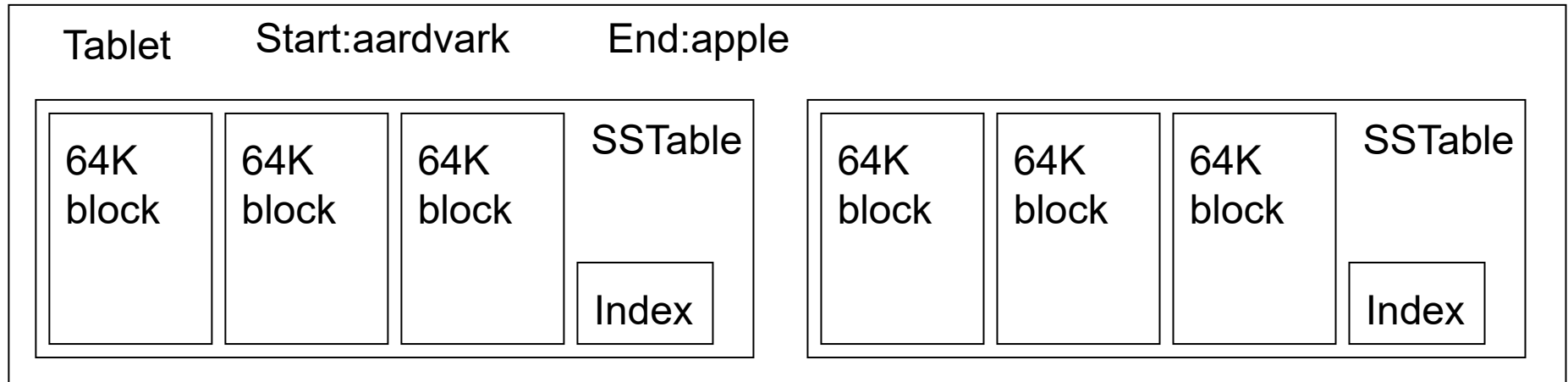
Timestamped
data versions

SSTable: Big Table Building Blocks



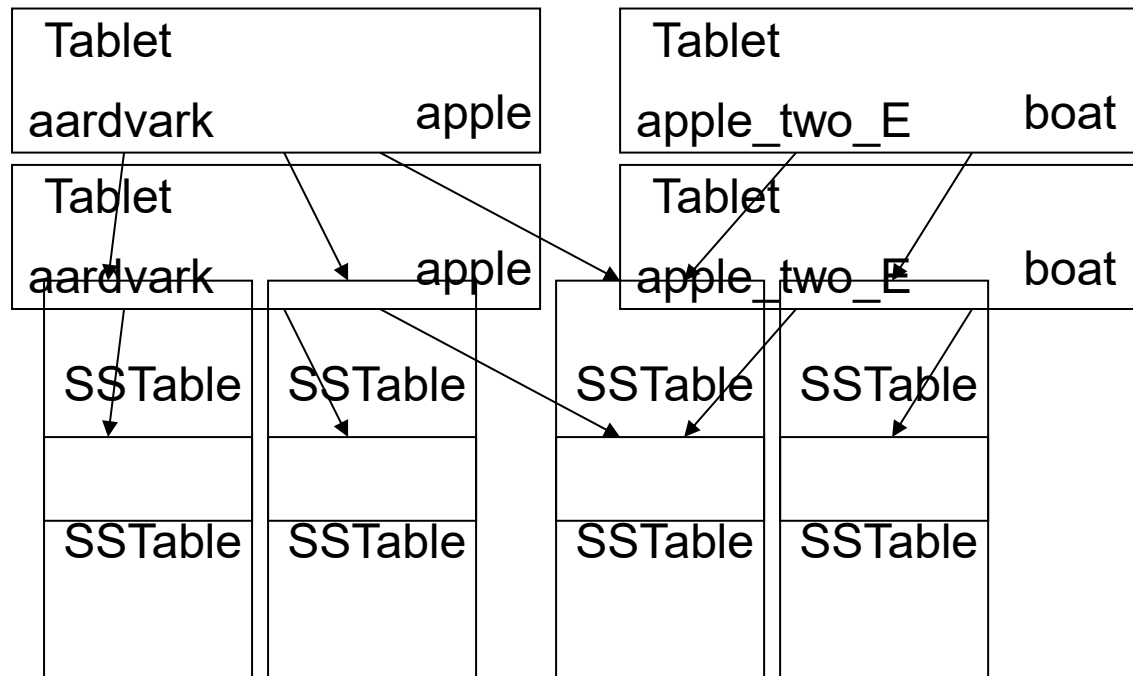
- Building blocks: SSTable
- Each SSTable has multiple blocks and a block index.
- Each block is a segment of the SSTable that stores a group of sorted keys and their associated values
- Index is of block ranges, not values
- Immutable, sorted file of key-value pairs
 - Once a block is written, no more updates possible!

Tablet: BigTable Building Blocks



- A Tablet is Built out of multiple SSTables
- A Tablet contains some range of rows of the table
- Bigtable maintains data in lexicographic order by row key. The row range for a table is dynamically partitioned

BigTable: BigTable Building Blocks



- Multiple tablets make up a table in BigTable
- SSTables can be shared
- Tablets do not overlap, SSTables can overlap in terms of the keys/rows they contain.

Tablet Servers: Big Table Building Blocks

- Tablet servers manage tablets,
 - Multiple tablets per Tablet server.
 - Each tablet lives at only one server
 - Each tablet is 100-200 MB
 - Tablet server splits tablets that get too big
- Master responsible for load balancing and fault tolerance

Master's Tasks

- Use Chubby to monitor health of tablet servers, restart failed servers
 - Tablet server registers itself by getting a lock in a specific chubby directory
 - Chubby gives “lease” on lock, must be renewed periodically
 - Server loses lock if it gets disconnected
 - Master monitors this directory to find which servers exist/are alive
 - If server not contactable/has lost lock, master grabs lock and reassigns tablets
 - GFS replicates data. Prefer to start tablet server on same machine that the data is already at

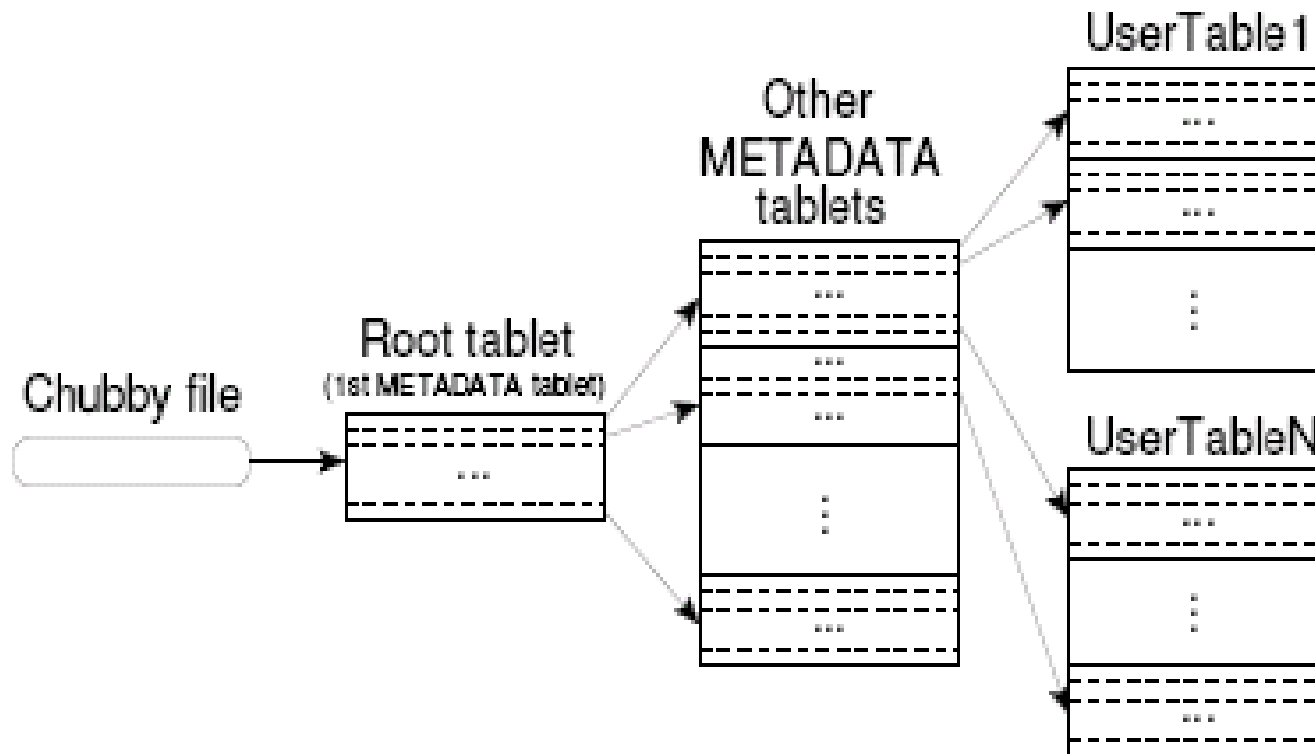
Master's Tasks (Cont)

- When (new) master starts
 - grabs master lock on chubby
 - Ensures only one master at a time
 - Finds live Tablet Servers (scan chubby directory)
 - Communicates with Tablet Servers to find assigned tablets
 - Scans metadata table to find all tablets
 - Keeps track of unassigned tablets, assigns them
 - Metadata root from chubby, other metadata tablets assigned before scanning.

Metadata Management

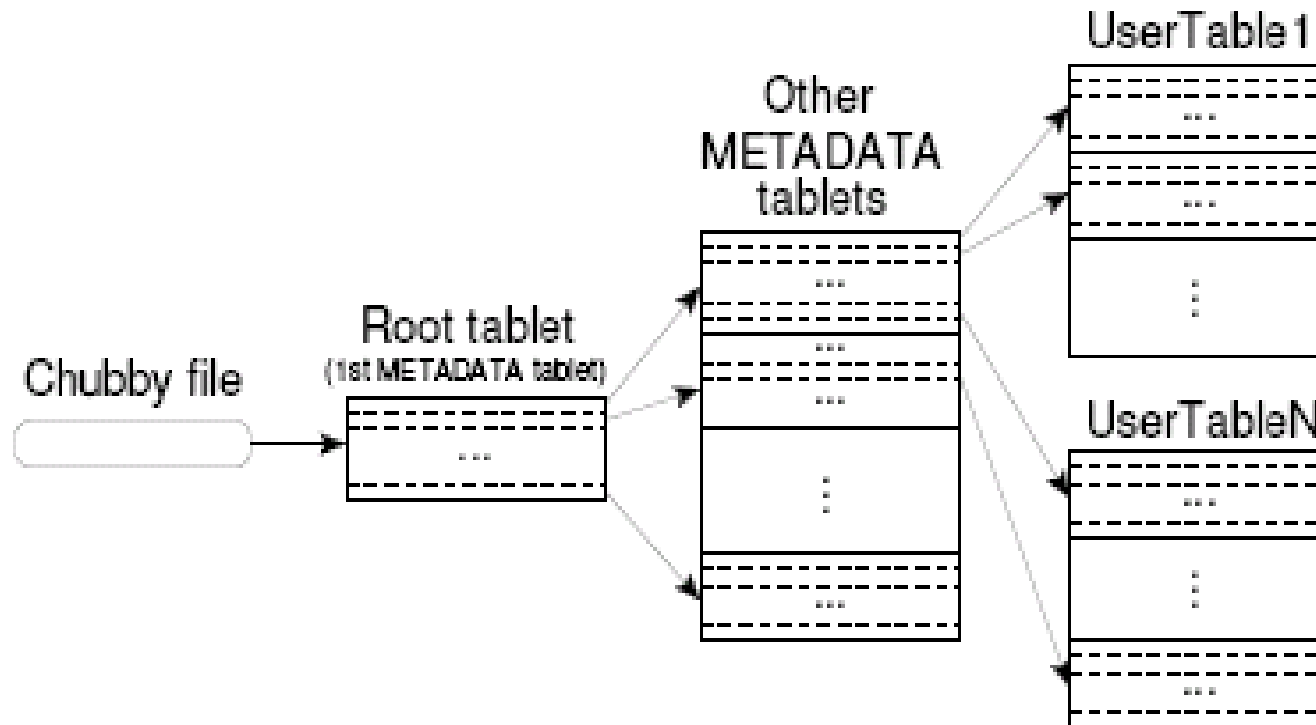
- Master handles
 - table creation and merging of tablet
- Tablet servers directly update metadata on tablet split, then notify master
 - lost notification may be detected lazily by master

Locating a Table/Tablet/SSTable



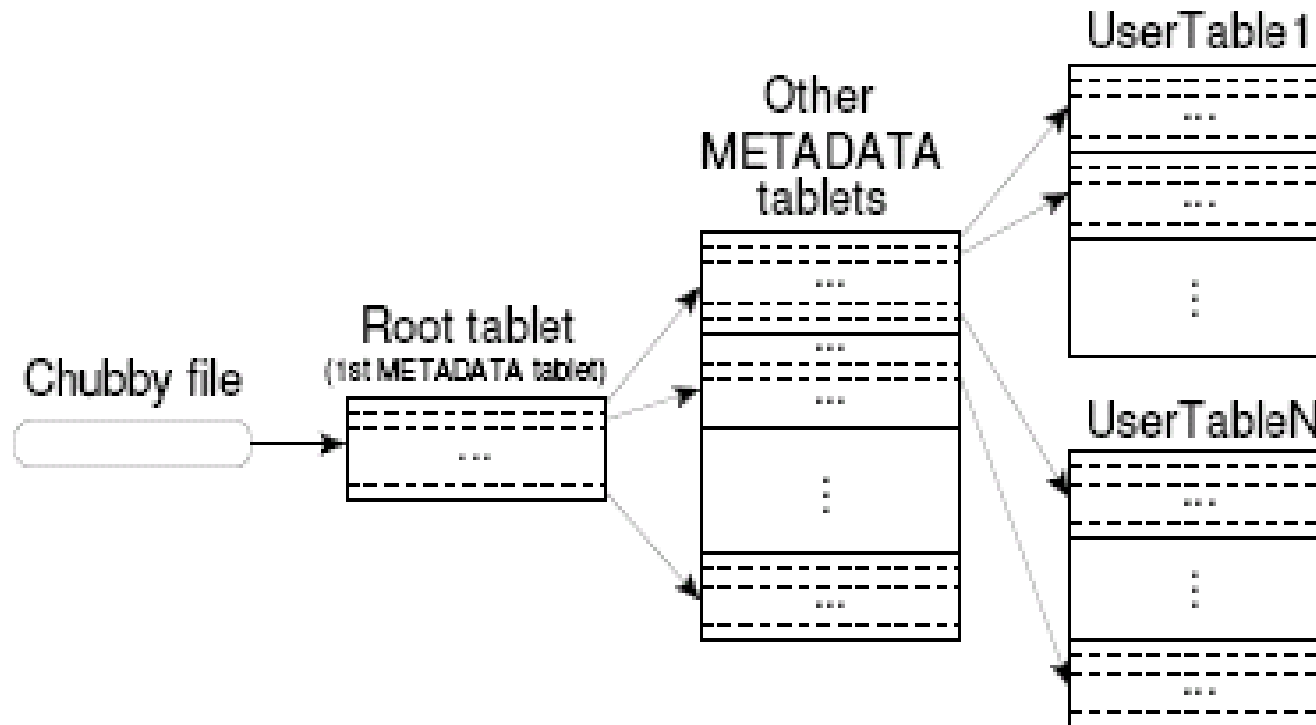
- BigTable uses a three-level hierarchy analogous to that of a B+ tree.
- The first level is a file stored in Chubby that contains the location of the root tablet.
- The root tablet contains the location of all tablets in a special METADATA table.

Locating a Table/Tablet/SSTable



- Each METADATA tablet contains the location of a set of user tablets.
- The METADATA table stores the location of a tablet under a row key that is an encoding of the tablet's table identifier and its end row.

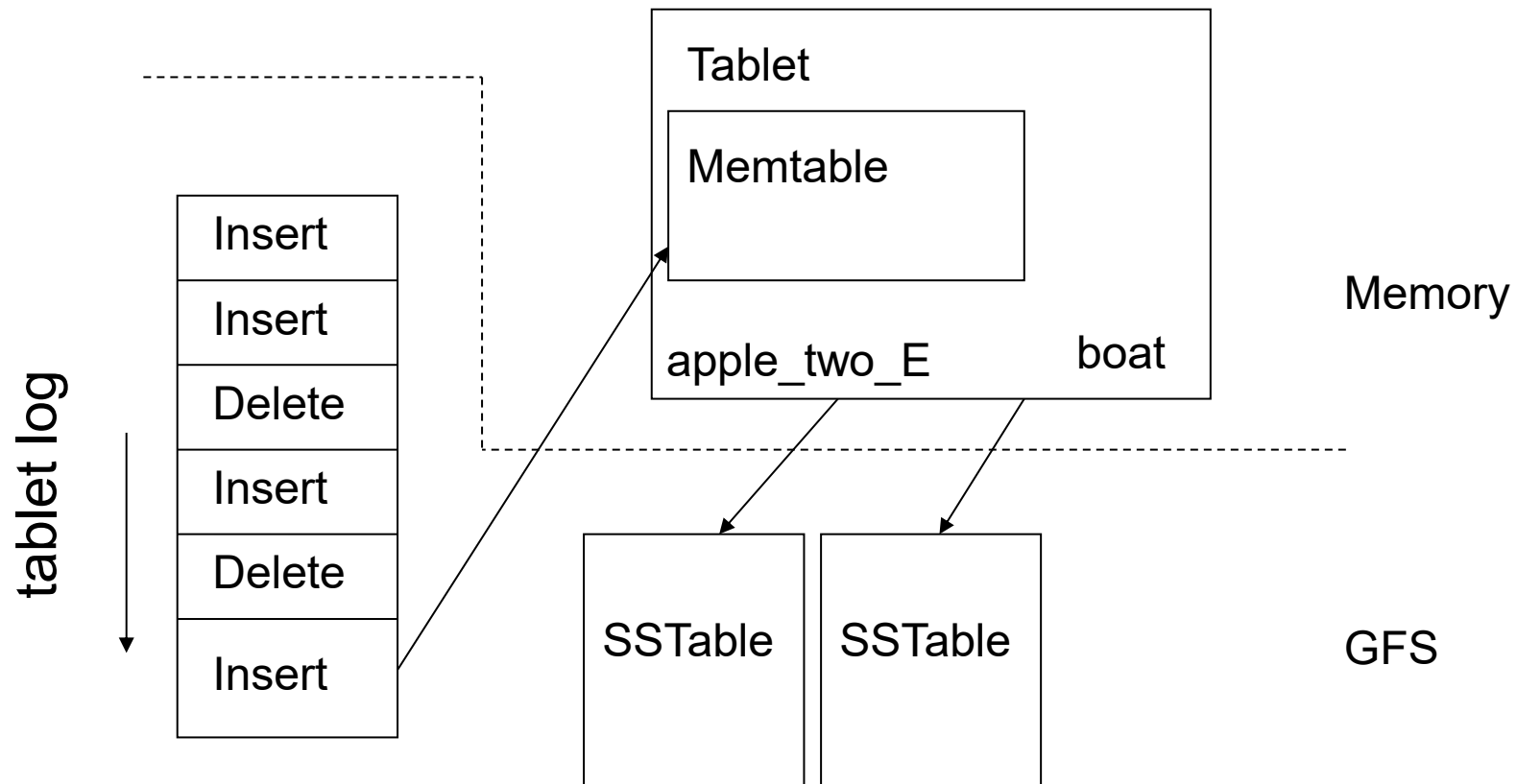
Locating a Table/Tablet/SSTable



- Each METADATA row stores approximately 1KB of data in memory.
- With a limit of 128 MB METADATA tablets, the three-level location scheme is sufficient to address 2^{34} tablets.

Editing a table

- Changes are logged, then applied to an in-memory memtable
 - May contain “deletion” entries to handle updates
 - Group commit on log: collect multiple updates before log flush



Immutability

- SSTables are immutable
 - simplifies caching, sharing across GFS etc
 - no need for concurrency control
 - SSTables of a tablet recorded in METADATA table
 - Garbage collection of SSTables done by master
 - On tablet split, split tables can start off quickly on shared SSTables, splitting them lazily
- Only memtable has reads and updates concurrent
 - copy on write rows, allow concurrent read/write

BigTable

- The data is always **maintained consistently**.
- Guarantee coming from using Chubby
- The model is naturally **partitioned** horizontally.
- But, **availability may be in question**.
- There could be brief downtimes, due to
 - Chubby not being available
 - network problems

Example Applications Using BigTable

- Goole Analytics:
 - Track various metrics such as traffic patterns, site-tracking reports, ...
- The raw click table (~200 TB) maintains a row for each end-user session.
- The row name is a tuple containing the website's name and the time at which the session was created.

Example Applications Using BigTable

- Goole Personalized Search:
- Records user queries and clicks across a variety of Google properties such as web search, images, and news.
- Users can browse their search histories to revisit their old queries and clicks, and they can ask for personalized search results based on their historical Google usage patterns.
- Personalized Search stores each user's data in Bigtable.
- Each user has a unique userid and is assigned a row named by that userid. All user actions are stored in a table.